Introduction to GitHub Actions - Exercises

Exercise 0: Initial Setup

- Create a fork or copy of the workshop repository into your own GitHub account https://github.com/ajaust/introduction-to-github-actions-edc26
  - Your copy or fork of the repository should be `public`.
  - If you create a fork, copy all branches. There is a `solution` branch.
- If you made a fork: Go to the settings of the fork and allow GitHub Actions to run on this fork
- Depending on how you want to work with the repository
  - Clone the repository to your laptop
  - Create a GitHub Codespace on the `main` branch

Exercise 1: Create Your First Workflow

Goal: Add a new workflow to the repository and inspect its output on the repository's website.

- Create a new folder `.github/workflows/`
- Create new workflow file `touch exercise1_workflow.yml` inside the `workflows/` directory with the following content

  ```yaml
  name: Workflow of exercise 1
  run-name: First workflow # Optional

  # Events triggering the workflow
  on: [push]

  # Group of jobs to run as part of the workflow
  jobs:
    greet:
      runs-on: ubuntu-latest
      steps:
        - run: echo "Hello world"
  ```

- Add file to repository, commit, and push the changes to the `main` branch

  In case you use a terminal:

  ```bash
  git add exercise1_workflow.yml
  git commit -m "Add exercise 1 workflow"
  git push
  ```

- Go to `Actions` tab of your repository.

  The workflow should run on GitHub (maybe is already finished).

- Inspect the output. There should be three steps

  1. `Set up job`
  2. `Run echo "Hello world"`
  3. `Complete job`

  Steps 1 and 2 are implicitly added by GitHub.

- Optional: Add/remove the `run-name`, commit & push the change and check how this influences the name of the workflow on the GitHub website.

Exercise 2: Multi-job workflows and failing jobs

Goal: Setting up a workflow with different runners, dependencies and inspecting failing jobs.

- Create a new branch `exercise2` and change to this branch, e.g., via `git checkout -b exercise2`
- Create a new workflow `exercise2_workflow.yml`.
- This workflow should run on the following events:
  - `push` to `main` branch
  - `pull_request` to `main` branch
  - `workflow_dispatch`
- The workflow should contain five jobs
  1. `ubuntu_job` that runs on `ubuntu-latest` and prints `Hello, I'm ubuntu-latest`
  2. `macos_job` that runs on `macos-latest` and prints `Hello, I'm macos-latest`
  3. `slow_job` that runs on `ubuntu-latest`, prints `Hello, I'm slow_job` and then sleeps for 2 minutes (120 seconds) and has a timeout of **1 minute**.
  4. `waiting_job` that runs on `ubuntu-latest`. It depends on `slow_job` and runs the command `ls -l`. This prints all files in the current directory.
- Add the new workflow to git, commit it and push it to GitHub.
- Check the Actions panel on GitHub and verify that the job did **not** run.
- Use your branch to open a pull request targetting main. You will see the Actions pop up at the bottom of pull request.
- Check the running workflows in the `Actions` panel of the repository. Inspect the output and verify that the `waiting_job` fails. Inspect the output of the failing job (`slow_job`). It should give you a hint why it is failing. The job `waiting_job` should not have started.
- Reduce the sleeping time of `slow_job` to 30 seconds and its timeout to 5 minutes. Push the changes to your repository. If you add the changes via a pull request, you should see that your newly added jobs are running. All jobs should run successfully and you should be able to see that `waiting_job` still waits for `slow_job`. Merge the PR afterwards.
- Go to the Actions panel in your repository and manually start you workflow.
- Check the output of `waiting_job` carefully. What files are being listed?
- At the end of the exercise, switch back to the main branch `git switch main`

Exercise 3: Matrix builds and conditionals

Goal: Setting up a workflow with different operating systems using GitHub's matrix builds while using predefined actions. Use `actionlint` to find mistakes early.

There is some very simple Python code in the repository, see `mycode.py`, that comes with some simple tests, see `test_mycode.py`.

- Create a new workflow `exercise3_workflow.yml`
- Carry out several tasks
  1. Update pip and install the dependencies needed by the Python code'

     ```
     python -m pip install --upgrade pip
     python -m pip install -r requirements-dev.txt
     ```

  2. Add a step that checks the style of the code `text python -m black mycode`
  3. Run the test by running `text python -m pytest test_mycode.py`
- The tests should be run on `windows-latest`, `macos-latest`, and `ubuntu-latest`.
- The tests should be run on Python version `3.12`, `3.13`, and `3.14`.
- Use `actionlint` to check your workflow file for correctness. If you don't make mistaked, introduce some explicit errors to see how the actionlint output looks like.
- Optional: Set up the workflow such that tests (`pytest`) are only executed if the style checks pass. The style checks should only run a single time using Python 3.14?

Exercise 4: Security

- Enable CodeQL in the repository settings. The setting is available under `Advanced Security`. Click on `Set up` -> `Default`.
- Check the Actions section in the repository. CodeQL should run as an Action/workflow.
- After the workflow finishes, you should see some warnings in the Security panel of the repository.
- Add explicit permission to your workflows. Start with dropping all permissions and check if the wofklow still succeeds. Only add permissions if necessary. Also check in the workflow logs that the permission are reduced.

  By default the "Set up job" step of a workflow had the following permissions:

  ```
  GITHUB_TOKEN Permissions
    Contents: read
    Metadata: read
    Packages: read
  ```

  Only permissions to read metadata should be left.
- Verify that the security warnings disappear from the Security overview.

Exercise 5: Creating artifacts and using Docker

Goal: Create downloadable PDFs of the workshop notes.

- Create a new workflow `exercise5_workflow.yml`. It should create PDFs out of the `exercise.md` file and upload it as an artifact.
- The can be converted to a PDF via the following command

  ```
  pandoc --template eisvogel exercises.md -o exercises.pdf
  ```

- Optional: Create a downloadable version of the slides. The slides can be converted to a PDF via the following command

  ```
  pandoc -t beamer slides.md  -o  slides.pdf
  ```

- Use the Docker container `pandoc/extra`. It contains everything needed to convert the markdown files into PDFs
- Check the `Run actions/upload-artifact@v7` step of the workflow. It should contain a link to the built PDF. Verify that the PDF looks correct.

  There is also an `Artifact` overview in the workflow summary. You might have to scroll down to see it.